



Les interruptions : en temps réel

Dans certaines applications, il est crucial que le système puisse répondre immédiatement à des événements qui se produisent à des moments imprévisibles. Par exemple, si un capteur détecte une température anormale ou si un bouton est pressé, le système doit pouvoir réagir immédiatement.

- **Événement asynchrone :**

Un événement **asynchrone** est un événement qui peut survenir à tout moment, indépendamment de l'état actuel du programme. Par exemple, dans un système embarqué, des événements peuvent être générés par des capteurs, des utilisateurs ou des communications réseau, et ces événements peuvent survenir à des moments qui ne sont pas prédéfinis.

Les interruptions permettent au processeur d'interrompre son traitement en cours pour gérer ces événements lorsqu'ils se produisent. Cela garantit que le système peut réagir sans délai, ce qui est essentiel dans de nombreuses applications critiques (comme les systèmes de contrôle, les systèmes de sécurité, etc.).



Les interruptions : en temps réel

Exemple: un système de contrôle de température dans un réfrigérateur. Si la température à l'intérieur du réfrigérateur dépasse un seuil critique, il est crucial que le système réagisse immédiatement pour éviter la détérioration des aliments. En utilisant une interruption, le système peut arrêter ce qu'il est en train de faire, activer le compresseur pour refroidir l'intérieur, et ensuite reprendre son traitement normal.

Les interruptions : en temps réel

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
```

```
#define TEMPERATURE_THRESHOLD 5 // Seuil critique en degrés Celsius
int current_temperature = 0;    // Température actuelle

// Fonction d'interruption pour gérer la surchauffe
void handle_overheat(int signum) {
    printf("Alerte : Température élevée ! Activation du compresseur...\n");
    // Activer le compresseur
    // ... (Code pour activer le compresseur)
}

void simulate_temperature_sensor() {
    // Simuler une variation de température
    current_temperature = rand() % 10; // Température aléatoire entre 0 et 9
    printf("Température actuelle : %d°C\n", current_temperature);

    if (current_temperature > TEMPERATURE_THRESHOLD) {
        raise(SIGUSR1); // Générer une interruption
    }
}
```

handle_overheat(int signum) : Cette fonction est appelée lorsque la température dépasse le seuil critique. Elle simule l'activation du compresseur.

simulate_temperature_sensor() : Cette fonction génère une température aléatoire et vérifie si elle dépasse le seuil critique. Si c'est le cas, elle génère une interruption en utilisant `raise(SIGUSR1)`.



Les interruptions : en temps réel

```
int main() {  
    // Enregistrer le gestionnaire d'interruption  
    signal(SIGUSR1, handle_overheat);  
  
    while (1) {  
        simulate_temperature_sensor(); // Lire la température  
        sleep(1); // Attendre 1 seconde avant la prochaine lecture  
    }  
    return 0;  
}
```

Dans la fonction main(), nous enregistrons le gestionnaire d'interruption pour SIGUSR1, puis entrons dans une boucle infinie qui simule la lecture de la température toutes les secondes.



Les interruptions : polling

Polling : C'est une méthode où le programme vérifie régulièrement (à intervalles fixes) l'état d'un événement ou d'une condition. Par exemple, un programme peut vérifier si un bouton a été pressé ou si des données sont disponibles sur un capteur.

Exemple de polling :

Supposons qu'on veut vérifier si un capteur de température a une nouvelle valeur :

```
while (1) {  
    if (sensorDataAvailable()) {  
        // Traiter les données du capteur  
    }  
    // Autres instructions...  
}
```

Dans cet exemple, le processeur est en train de vérifier en permanence si des données sont disponibles. Cela peut consommer beaucoup de temps CPU, même si rien ne se passe.



Les interruptions : polling

Pourquoi utiliser une interruption à la place :

1. Réagir en temps réel : Les interruptions permettent au processeur de réagir immédiatement lorsqu'un événement se produit (comme un bouton pressé ou un signal d'un capteur). Au lieu de vérifier sans cesse, le processeur peut continuer à exécuter d'autres tâches jusqu'à ce qu'une interruption se produise.

2. Économiser le temps d'exécution : Utiliser des interruptions permet d'éviter que le processeur ne passe du temps à vérifier des conditions qui pourraient ne jamais se produire. Cela libère des ressources pour d'autres tâches, rendant le programme plus efficace et réactif.

En résumé :

- En utilisant **polling**, le processeur est continuellement occupé à vérifier des conditions, ce qui peut entraîner une utilisation inefficace des ressources.
- En utilisant des **interruptions**, le processeur ne s'arrête que lorsqu'un événement important se produit, ce qui permet d'économiser du temps d'exécution et d'améliorer la réactivité du système.